

Relazione 3

Federico Becchi

13 luglio 2026

1 Calcolo PiGreco

L'obiettivo del programma è calcolare il pigreco, vengono utilizzati due metodi: f1, f2. Si sta utilizzando la funzione f2.

Esecuzione file `cpi.slurm`

```
N=1000000000

gcc cpi.c -O2 -o cpi -lm
./cpi -n $N

gcc cpi+amdahl.c -O2 -o cpi+amdahl -lm
./cpi+amdahl -n $N -s 100000
```

Il file `cpi.slurm.2510099.out` è il file di output:

```
[federico.becchi@ui01 cpi]$ cat cpi.slurm.2510099.out
#Inter error time hostname
1000000000, 1.04760645e-12, 1.01459, wn23
#Inter error tp tnp tp1+tnp hostname
1000000000, 6.33271213e-13, 0.55609, 0.10016, 0.65625, wn23
[federico.becchi@ui01 cpi]$
```

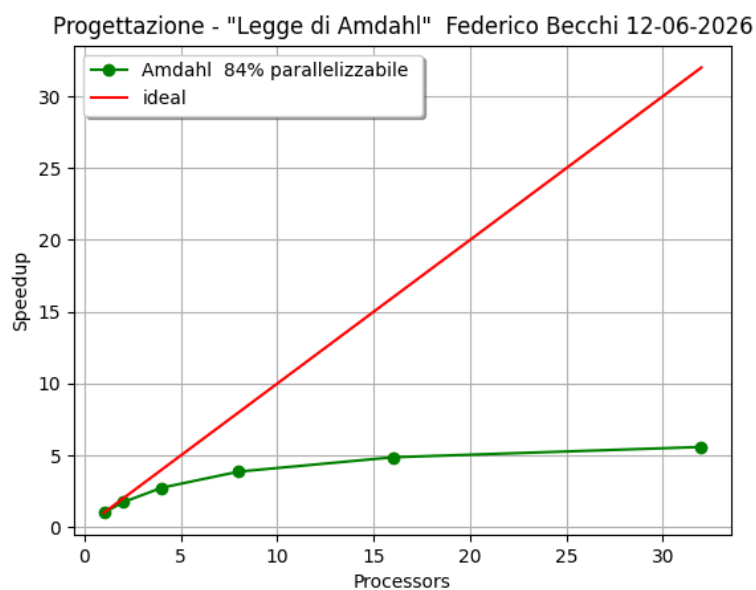


Figura 1: `cpi+amdahl.png`

2 Heat

`heat.c` vuole simulare la propagazione del calore con i bordi e il centro mantenuti a temperatura costante (1.0).

```
gcc -O2 heat.c -o heat

./heat -d -r 20 -c 20 (numero di righe 20, numero di colonne 20)

./heat 1>heatmap.txt 2>heat.csv
```

`heatmap.txt` è la matrice finale dopo le iterazioni.

`heat.csv` è la stringa composta da:

```
NX,NY,MAX_ITER,time, Jtime (jacobi update)
512, 512, 1000, 0.364, 0.329
```

heat_scaling.slurm

Vuole fare uno scaling test di `heat`, per vedere come cambiano i tempi di esecuzione all'aumentare della dimensione della matrice.

```
(venv) [federico.becchi@ui01 heat]$ cat heat_scaling.csv
nx, ny, iter, time, jtime
512, 512, 1000, 0.379, 0.344
758, 758, 1000, 1.044, 1.018
1024, 1024, 1000, 2.737, 2.598
2048, 2048, 1000, 13.909, 12.357
```

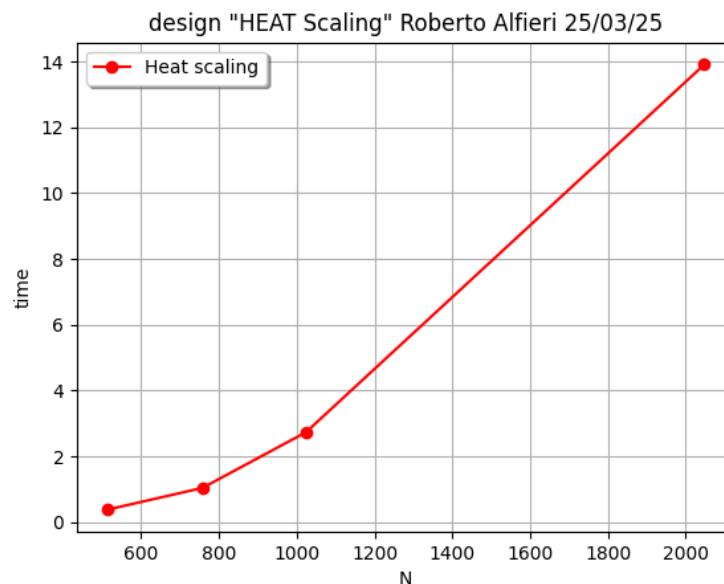


Figura 2: `heat_scaling.png`

heat-pg.slurm

`gcc -O2 -pg heat.c -o heat-pg`: si attivano i flag `-pg` per mettere le info di profiling.

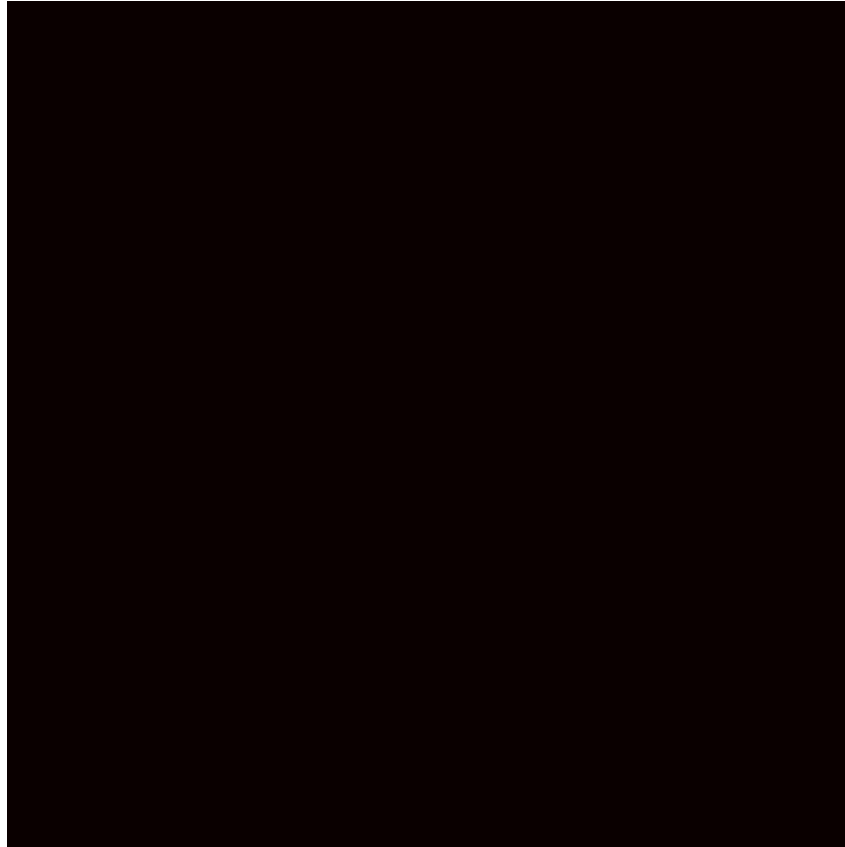


Figura 3: heatmap-animation.gif (primo frame, il PDF non supporta le GIF animate)

```
% cumulative self self total
time seconds seconds calls Ts/call Ts/call name
80.41 15.70 15.70 Jacobi_Iterator_CPU
19.31 19.47 3.77 Init_center
0.46 19.56 0.09 copy_cols
0.31 19.62 0.06 print_colormap
```

3 Factorize

Il programma fattorizza a forza bruta, ovvero genera una sequenza di numeri primi p (dispari) e verifica il resto della divisione Modulo/ p .

Esecuzione di `time` con modulo a 68 bit di cui 8 bit di address (2^8 blocchi disponibili) e ogni blocco ha 2^{26} numeri dispari da testare.

```
sbatch factorize.slurm
```

Questo è il modulo usato: MODULUS=B81915BC0A2222F4B

```
processed block 221 in 2.2 sec. with result 1
FOUND: 375C8B8B3

real 2m41.694s
user 2m41.307s
sys 0m0.002s
```

3.1 Decomposizione del dominio

La decomposizione del dominio è un vettore di $[1, 2^{primebit}]$ numeri. Il sorgente `factorize.c` lo scompone già in blocchi continui di dimensione $2^{block_dim_bit}$, il numero totale di blocchi è $2^{block_addr_bit}$.

Si tratta di una decomposizione a blocchi continua ove ogni blocco è un intervallo di numeri consecutivi.

Quello applicato è un bilanciamento a grana grossa in cui un numero di blocchi eseguono la ricerca del fattore. La ricerca all'interno del blocco non è prevedibile e sbilanciata.

3.2 Legge di Amdahl applicata a questo problema

Come parte parallelizzabile si assume la sezione di ricerca del fattore all'interno del blocco. Come parte seriale quella antecedente all'esecuzione della ricerca e successiva al blocco.

Speedup ideale Il fattore dovrebbe essere sempre nell'ultimo blocco per ottenere un incremento lineare rispetto al numero di thread/processi.

Speedup reale misurato L'overhead è trascurabile nelle parti seriali, tuttavia una parallelizzazione non comporta un reale aumento delle performance a livello di tempo.

3.3 Overhead: memoria condivisa vs distribuita

Memoria Condivisa L'overhead si ottiene nella sincronizzazione nell'accesso alle variabili condivise M, F . Invece il load balancing si può ottenere con uno scheduling dinamico, ovvero appena un thread finisce l'esecuzione del suo job può essere assegnato ad un blocco non ancora processato.

Memoria distribuita Non si pone il problema di race condition sull'accesso a M, F . Si ottiene un overhead di comunicazione quando un task si ferma e notifica di aver trovato il fattore. Questo comporta che ogni job periodicamente dovrà controllare la ricezione del messaggio di fine per poi ritornare alla ricerca del fattore.